

🎯 本章目标

掌握 C++ 的异常机制与栈展开,理解异常安全与 noexcept;学会用 std::thread 创建线程、用 mutex 与 atomic 保护共享数据、用条件变量与 async 组织并发;识别数据竞争、死锁等并发陷阱,把第8章的单线程服务端升级为多线程版本。

📦 前置知识

熟悉 RAII(第5章)与 lambda(第4章);了解函数声明与变量作用域(第1、4章)。本章代码基于 C++17,编译多线程代码需要链接线程库(pthread 或 Windows 线程库)。

§9.1 异常机制与栈展开	§9.2 异常安全与 noexcept	§9.3 std::thread 线程
§9.4 互斥与原子	§9.5 条件变量与 async	§9.6 并发设计原则

§9.1 异常机制与栈展开

C 的错误处理是返回值加 errno:每个函数都要检查返回值,漏一次检查错误就静默扩散;setjmp/longjmp 能跳转,但跳过中间函数的清理代码,资源泄漏。C++ 的异常把"错误路径"从正常路径中分离:throw 抛出异常对象,异常沿着调用栈向上传播,直到匹配的 catch 捕获;传播过程中,栈上所有对象依次析构,这就是栈展开。展开的每一步都调用析构函数,RAII 类(第5章)因此保证"异常路径上资源同样被释放"——这是 C 的 longjmp 做不到的。

写 catch 的三条纪律:第一,按引用捕获 catch (const std::exception& e),按值捕获会把派生异常切片成基类,丢失信息;第二,捕获顺序从具体到宽泛,std::bad_alloc 要在 std::exception 之前;第三,不想处理的异常就让它继续传播,或者用 catch(...) 兜底,记录日志后再重新抛出。真正不知道怎么办时,让程序按默认规则终止(std::terminate),好过带着半损坏状态继续运行。

C 的写法	C++ 的写法	说明
if (ret < 0) { 处理错误 }	try { ... } catch (const X& e)	C 检查点散落,C++ 错误路径集中
errno 全局变量,易被覆盖	异常对象携带完整信息	C++ 异常类型可含上下文
longjmp 跳过清理代码	栈展开自动析构	C++ 与 RAII 配合,资源不泄漏

清单 9-1 异常基础与栈展开

```

#include <iostream>
#include <stdexcept>
#include <string>

class Guard {                                // RAII:观察析构时机
public:
    explicit Guard(const std::string& n) : name(n) {}
    ~Guard() { std::cout << "析构 " << name << "\n"; }
private:
    std::string name;
};

double divide(double a, double b) {
    Guard g("divide 作用域");
    if (b == 0.0) throw std::runtime_error("除数不能为零");
    return a / b;
}

int main() {
    try {
        Guard g("main 作用域");
        std::cout << divide(10, 0) << "\n";
    } catch (const std::runtime_error& e) {    // 按引用捕获
        std::cout << "捕获:" << e.what() << "\n";
    }
    return 0;    // 输出:析构 divide 作用域 → 捕获 → 析构 main 作用域
}

```

⚠ 易错点 1:按值捕获导致切片

`catch (std::exception e)` 会先把派生异常拷贝成基类再捕获,子类携带的额外信息全部丢失,虚函数调用也退化成基类版本。一律写 `catch (const std::exception& e)`,既避免切片又避免拷贝开销。

§9.2 异常安全与 noexcept

异常安全的级别有三档:基本保证(出错后对象保持有效状态)、强保证(操作要么完全成功要么像没发生过)、无异常保证(绝不抛出)。提供强保证的经典手段是"先构造副本,再一次性交换":把会抛异常的操作全部放在临时对象上,最后用不抛异常的 `swap` 提交——即使中途抛异常,原对象纹丝不动。RAII 是异常安全的地基:锁、内存、文件都在析构中释放,展开路径上不会泄漏。

`noexcept` 声明"这个函数不抛异常":函数内部若真的抛出,程序直接调用 `std::terminate` 终止。它既是承诺也是优化——编译器知道无需准备展开路径。移动构造与移动赋值必须声明 `noexcept`,否则 `vector` 扩容时保守地退回拷贝,性能退化;`swap`、析构函数、默认构造函数等"应当成功"的操作也习惯性标记 `noexcept`。判断口诀:不涉及资源获取、不调用可能抛出函数、失败也无妨的操作,都可以 `noexcept`。

清单 9-2 强异常保证与 noexcept

```

#include <iostream>
#include <vector>
#include <string>
#include <utility>

class Config {
public:
    void set(const std::string& k, const std::string& v) {
        std::vector<std::pair<std::string, std::string>> tmp = kv;
        for (auto& p : tmp) {
            if (p.first == k) { p.second = v; goto done; }
        }
        tmp.emplace_back(k, v);          // 可能抛异常:发生在临时对象上
    done:
        kv.swap(tmp);                    // 不抛异常的提交:强保证
    }
    std::string get(const std::string& k) const {
        for (const auto& p : kv) if (p.first == k) return p.second;
        return "";
    }
private:
    std::vector<std::pair<std::string, std::string>> kv;
};

class Token {                          // 移动必须 noexcept 的范例
public:
    explicit Token(std::string v) : v(std::move(v)) {}
    Token(Token&& o) noexcept : v(std::move(o.v)) {}
    Token& operator=(Token&& o) noexcept {
        v = std::move(o.v);
        return *this;
    }
    const std::string& value() const { return v; }
private:
    std::string v;
};

int main() {
    Config c;
    c.set("name", "张三");
    c.set("age", "18");
    std::cout << c.get("name") << " " << c.get("age") << "\n";
    std::vector<Token> v;
    v.push_back(Token("甲"));           // 扩容时用移动,无拷贝
    return 0;
}

```

§9.3 std::thread 线程

C 的并发工具是 pthread 一族:创建、传参、返回值全凭手工约定。C++11 的 std::thread 把线程包装成对象:构造函数直接接受可调用对象(函数、lambda、函数对象),参数跟着传;join() 等待线程结束,detach() 让线程独立运行。生命周期纪律:thread 对象析构时若线程还在运行且未 join/detach,程序直接终止——要么 join,要么 detach,没有第三种选择。

传参的两个细节:线程函数的引用参数必须用 `std::ref` 显式包装,否则按值拷贝一份,修改不生效;lambda 捕获与线程一起使用时,捕获的引用指向的变量必须活得比线程长——`detach` 之后尤其危险,线程还在跑,局部变量已经销毁,访问就是未定义行为。第8章的服务端要并发,最小改动就是每接一个连接,开一个线程去处理,处理完 `join` 或 `detached` 线程自己退出。

C 的写法	C++ 的写法	说明
<code>pthread_create(&t, ...)</code>	<code>std::thread t(f, args)</code>	C 填结构体传参,C++ 直接给参数
<code>pthread_join / pthread_detach</code>	<code>t.join() / t.detach()</code>	同语义,成员函数更直观
返回结果靠全局变量	<code>std::async + future</code> 取回	C++ 结果随对象返回

清单 9-3 线程的创建与生命周期

```
#include <iostream>
#include <thread>
#include <vector>

void work(int id, int& out) {
    out = id * id;           // 通过引用把结果写出去
}

int main() {
    std::vector<std::thread> pool;
    std::vector<int> results(4, 0);

    for (int i = 0; i < 4; ++i) {
        pool.emplace_back(work, i, std::ref(results[i])); // 引用要 std::ref
    }
    for (auto& t : pool) t.join();           // 逐个等待,不 join 就析构会终止程序

    for (int x : results) std::cout << x << " ";    // 0 1 4 9
    std::cout << "\n";

    // lambda 线程 + detach 的危险示范:注释掉看程序行为
    // int local = 42;
    // std::thread([&local] { std::cout << local << "\n"; }).detach();
    return 0;
}
```

⚠ 易错点 2:detach 后访问已销毁的局部变量

`std::thread([&x]{ use(x); }).detach();` 线程独立运行,函数返回后 `x` 已销毁,线程访问 `x` 是未定义行为——可能碰巧能跑,也可能随机崩溃。规则:detach 的线程只能访问它自己的拷贝与全局/堆上生命周期明确的数据,引用局部变量的线程必须 `join`。

§9.4 互斥与原子

多个线程同时读写同一份数据就是数据竞争,数据竞争是未定义行为——这不是“可能出错”,而是程序任何行为都可能。保护数据的手段有二:互斥锁与原子操作。`std::mutex` 提供 `lock/unlock`,但手工配对和 C 的 `fopen/fclose` 一样容易漏;正确用法是 RAII 包装:`std::lock_guard` 构造时加锁、析构时解锁,异常路径也安全;`std::unique_lock` 额外支持手动解锁与延迟加锁,是条件变量(下节)的搭档。工程准则:锁的粒度越小越好,持锁时间越短越好,绝不在持锁时做 I/O 或等待。

std::atomic 提供无锁的原子操作:对一个 atomic 整数的读写是线程安全的,底层是 CPU 的原子指令,比加锁轻量得多,适合计数器、标志位、共享的简单状态。注意"无锁"不等于"免同步":两个线程先读后写同一个 atomic,仍然需要顺序语义(compare_exchange 等),真正的复杂并发还是交给锁。死锁的典型成因是锁顺序不一致:线程 A 持有锁1等锁2,线程 B 持有锁2等锁1,互相等待永不释放。预防口诀:全项目统一加锁顺序,或者用 std::scoped_lock 一次锁多把。

C 的写法	C++ 的写法	说明
pthread_mutex_lock/unlock 成对	std::lock_guard 自动解锁	RAII 保证异常路径也解锁
__sync_fetch_and_add 或汇编	std::atomic<int> cnt	可移植的原子操作
锁顺序靠自觉	scoped_lock 一次锁多把	工具辅助,防死锁于设计

清单 9-4 互斥锁与原子计数器

```
#include <iostream>
#include <thread>
#include <mutex>
#include <atomic>
#include <vector>

std::mutex mtx;           // 保护 balance
double balance = 0.0;     // 被锁保护的共享数据
std::atomic<int> ops{0};   // 无锁计数器

void deposit(double amount, int times) {
    for (int i = 0; i < times; ++i) {
        {
            std::lock_guard<std::mutex> lock(mtx); // RAII 锁
            balance += amount;                     // 持锁改数据
        }                                          // 解锁
        ++ops;                                    // 原子自增
    }
}

int main() {
    std::vector<std::thread> ts;
    ts.emplace_back(deposit, 1.0, 10000);
    ts.emplace_back(deposit, 2.0, 10000);
    for (auto& t : ts) t.join();
    std::cout << "余额 " << balance << ",操作数 " << ops
                << "\n"; // 30000 与 20000,顺序无关
    return 0;
}
```

§9.5 条件变量与 async

条件变量解决"等待某个条件成立":生产者准备好数据后 notify_one/notify_all 唤醒等待者,消费者在 wait 上挂起等待。使用套路固定:wait 必须配合 unique_lock,且用带谓词的版本 wait(lk, pred)——它循环检查谓词,自动处理"虚假唤醒"(条件变量可能被莫名唤醒,谓词循环是唯一可靠写法)。没有谓词的裸 wait 是并发 bug 的温床。

大多数并发需求其实用不着直接碰线程与条件变量:std::async 接受一个任务,返回 std::future,调用 future.get() 取结果并等待完成。async 决定在哪个线程执行交给实现,任务返回值直接送到 future,异常也会随 future 传递——get() 会把

任务里的异常重新抛出。这个"任务 + 未来结果"的模型比手写线程加共享变量安全得多,是"并发优先方案"。future 家族的底层是 promise: promise 承诺将来提供值, set_value 兑现, set_exception 报告错误, 适合跨线程传值。

清单 9-5 条件变量与 async

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <future>
#include <chrono>

std::mutex m;
std::condition_variable cv;
bool ready = false;           // 谓词:数据就绪

void producer() {
    std::this_thread::sleep_for(std::chrono::milliseconds(50));
    {
        std::lock_guard<std::mutex> lk(m);
        ready = true;         // 修改谓词必须持锁
    }
    cv.notify_one();          // 唤醒一个等待者
}

double slowAdd(double a, double b) {
    std::this_thread::sleep_for(std::chrono::milliseconds(30));
    return a + b;
}

int main() {
    // 条件变量:等待就绪
    std::thread t(producer);
    {
        std::unique_lock<std::mutex> lk(m);
        cv.wait(lk, [] { return ready; }); // 带谓词,处理虚假唤醒
        std::cout << "数据已就绪\n";
    }
    t.join();

    // async:任务 + 未来结果
    std::future<double> f1 = std::async(std::launch::async, slowAdd, 1, 2);
    std::future<double> f2 = std::async(std::launch::async, slowAdd, 3, 4);
    std::cout << "和:" << f1.get() + f2.get() << "\n"; // 3 与 7 并行计算
    return 0;
}
```

⚠ 易错点 3:持锁时等待,死锁的温床

线程 A 持锁等条件变量,线程 B 需要同一把锁才能让条件成立——两边互相等,永远等不到。规则:等待条件、做 I/O、调用其他需要锁的代码,都不能发生在持锁期间;实在要等,先把锁释放(这正是 unique_lock 存在的意义)。

§9.6 并发设计原则

并发代码的正确性靠设计,不靠调试:数据竞争是未定义行为,能复现的竞争是运气,复现不了的竞争才是常态。四条原则:第一,最小共享——能不共享就不共享,每个线程只碰自己的数据,结果最后合并;第二,共享必保护——共享数据必须与锁或原子成对出现,注释里写明"由 mtx 保护";第三,锁内不做等待——持锁时绝不调用可能阻塞的 I/O 或再等条件变量,否则死锁概率指数上升;第四,用更高级的工具——async 与 future 能解决八成的"算完取结果",线程加互斥留给真正的长期运行任务。

性能上还要破除两个误解:线程不是越多越好——创建销毁有开销,线程数接近 CPU 核心数时收益封顶,再多就是切换成本;加锁也有开销,热路径上的高频小操作优先用 atomic,实在要用锁,保持临界区极短。第8章的单线程服务端升级并发的标准路线:accept 循环不变,每接一个连接,把处理扔给线程池(见实验),连接多时再用事件驱动模型。

并发与异常速查表

概念	语法示例	要点
抛出异常	<code>throw std::runtime_error("msg");</code>	沿调用栈传播,栈展开逐层析构
捕获异常	<code>catch (const std::exception& e)</code>	必须按引用,防切片
不抛异常	<code>void f() noexcept</code>	内部抛出直接 terminate
强异常保证	副本修改 + swap 提交	失败时对象保持原状
创建线程	<code>std::thread t(f, arg1, arg2)</code>	参数直接传,引用需 <code>std::ref</code>
等待线程	<code>t.join()</code>	析构前必须 join 或 detach
独立线程	<code>t.detach()</code>	只能访问自持数据
互斥锁	<code>std::lock_guard<std::mutex> lk(m)</code>	RAII,异常路径也解锁
手动锁	<code>std::unique_lock<std::mutex></code>	条件变量的搭档
原子操作	<code>std::atomic<int> cnt</code>	无锁,适合计数器标志位
条件变量	<code>cv.wait(lk, pred) / notify_one</code>	必须带谓词,防虚假唤醒
异步任务	<code>std::async(f, args)</code>	返回 future, <code>get()</code> 取结果
未来结果	<code>std::future<T></code>	结果与异常随 future 传递
数据竞争	多线程同读写无保护	未定义行为,靠设计避免

自测题

- Q 1. 栈展开是什么?RAII 为什么与异常是天生一对?
- Q 2. 按值捕获异常为什么是错的?catch 的顺序有什么讲究?
- Q 3. thread 析构时线程还在运行会怎样?join 与 detach 各自适合什么场景?
- Q 4. 数据竞争为什么是未定义行为?mutex 与 atomic 怎么选?
- Q 5. 条件变量的 wait 为什么要带谓词?async 相比手写线程有什么优势?

✅ 参考答案

1. 异常抛出后,调用栈上每层函数的作用域对象依次析构,再逐层向外找匹配的 catch,这个过程叫栈展开。RAII 对象在展开中必然析构,资源自动释放,所以异常路径与正常路径同样安全,C 的 longjmp 没有这个机制。
2. 按值捕获先把派生异常切片成基类,类型信息与虚函数行为丢失;按引用捕获零拷贝且保留完整类型。捕获顺序从具体到宽泛:先捕获派生异常,最后才是 std::exception 与 catch(...),否则派生异常永远轮不到。
3. 线程对象析构时若线程仍可 join,程序直接 std::terminate。join 适合"等它干完"的场景(任务分发、汇总结果);detach 适合"后台自生自灭"的场景,但线程必须只访问自己的拷贝或生命周期明确的数据。
4. 数据竞争意味着程序行为在语言层面失去定义,任何结果都算"正确",没法调试。选择:m 变量:原子操作适合计数器、标志位等简单状态,开销小;复杂数据结构的修改必须用互斥锁保护。
5. 条件变量可能被虚假唤醒,裸 wait 返回不代表条件成立;带谓词的 wait 会循环检查,条件不满足就继续等。async 把"启动任务、取回结果、传递异常"全部封装,调用方不用管理线程生命周期与共享数据,出错面小得多。

实验:多线程账本系统

本实验把本章知识组装成并发记账系统:多个线程同时存款,互斥锁保护余额,原子计数统计操作笔数,条件变量通知全部完成,async 并行计算汇总,异常在任务中抛出后由 future 传递。代码分三部分,覆盖点已在注释中标明。

实验 9•第一部分:共享账本(覆盖:mutex、lock_guard、atomic、数据竞争防护)


```

#include <iostream>
#include <thread>
#include <mutex>
#include <atomic>
#include <vector>
#include <condition_variable>
#include <future>
#include <chrono>

class Ledger {                                // 覆盖:共享数据与锁成对出现
public:
    void deposit(double amount) {
        std::lock_guard<std::mutex> lk(mtx);    // 覆盖:RAII 锁
        balance += amount;                    // 临界区极短
    }
    double total() const {
        std::lock_guard<std::mutex> lk(mtx);
        return balance;
    }
private:
    mutable std::mutex mtx;                    // 保护 balance
    double balance = 0.0;
};

Ledger ledger;
std::atomic<int> opCount{0};                  // 覆盖:原子计数器,无锁
std::mutex doneMtx;
std::condition_variable doneCv;
bool allDone = false;                        // 谓词:全部线程完成

void worker(int id, int times) {              // 覆盖:线程函数 + 循环存款
    try {
        for (int i = 0; i < times; ++i) {
            ledger.deposit(1.0);
            ++opCount;
            if (i == times / 2 && id == 2) {
                throw std::runtime_error("模拟中途故障");    // 覆盖:异常
            }
        }
    } catch (const std::exception& e) {
        std::cerr << "线程 " << id << " 故障: " << e.what() << "\n";
    }
    {
        std::lock_guard<std::mutex> lk(doneMtx);
        if (--remaining == 0) allDone = true;    // 覆盖:谓词更新持锁
    }
    doneCv.notify_all();                        // 覆盖:通知等待者
}

```

实验 9•第二部分:async 并行任务(覆盖:std::async、future、get 取结果、异常随 future 传递)

```

// 覆盖:async 任务—每个区块的账目各自算,结果并行取回
double auditBlock(int id, int blocks) {
    double sum = 0.0;
    for (int i = 0; i < 10; ++i) {
        sum += ledger.total() * 0.001;    // 只读汇总,不持锁写
    }
    if (id == blocks - 1) {
        throw std::runtime_error("审计故障");    // 覆盖:任务内抛异常
    }
    return sum;
}

std::future<double> auditAll(int blocks) {
    std::vector<std::future<double>> fs;
    for (int i = 0; i < blocks; ++i) {
        fs.push_back(std::async(std::launch::async, auditBlock, i, blocks));
    }
    double total = 0.0;
    for (auto& f : fs) {
        try {
            total += f.get();    // 覆盖:get 等待并取结果
        } catch (const std::exception& e) {
            std::cout << "审计任务异常: " << e.what() << "\n";
        }
    }
    return std::async(std::launch::async, [total] { return total; });
}

```

实验 9•第三部分:main 调度(覆盖:join、条件变量等待、谓词、整体组合)

```

int main() {
    std::vector<std::thread> workers;    // 覆盖:线程容器
    for (int i = 0; i < 3; ++i) {
        workers.emplace_back(worker, i, 2000);    // 3 个线程各存 2000 次
    }
    {
        std::unique_lock<std::mutex> lk(doneMtx);
        doneCv.wait(lk, [] { return allDone; });    // 覆盖:带谓词的等待
    }
    for (auto& t : workers) t.join();    // 覆盖:join 等待线程结束

    std::cout << "存款完成,余额 " << ledger.total()
              << ",操作 " << opCount << " 笔\n";

    auto fut = auditAll(3);    // 覆盖:async 并行审计
    std::cout << "审计汇总 " << fut.get() << "\n";
    return 0;
}

```

实验覆盖点核对:mutex 与 lock_guard(Ledger::deposit 临界区)、atomic 计数器(opCount)、条件变量与谓词(等待全部线程完成)、thread 容器与 join、async/future/异常传递(auditAll 收集各任务结果,任务异常在 get 时重新抛出并被捕获)、lambda 任务(返回汇总的 async)。运行观察:余额恒为 6000(3 线程各 2000 次,故障线程只影响自身循环),操作数恒为 6000,审计异常被捕获后程序正常结束。扩展练习:把 remaining 改成原子变量对比行为;或给 worker 增加随机延迟,观察条件变量唤醒的时机。

工程建议 1:共享数据用类封装

把"数据 + 保护它的锁"装进同一个类(Ledger 模式),锁不出类,数据不出类,外部只能通过加锁的接口访问。这样"谁保护什么"一目了然,代码评审时不用满文件找裸共享变量。

工程建议 2:优先 `async`,线程交给框架

新写并发逻辑先问:能不能用 `std::async` 表达?任务返回结果、互相独立——`async` 一行搞定,还免费获得异常传递。只有长期运行的循环、事件循环、线程池等才值得亲手管理 `std::thread`;线程池直接采用成熟库(如 TBB),别从零造。

本章小结

异常机制把错误路径从正常路径分离,栈展开与 RAII 让资源释放成为语言保证;`noexcept` 与强异常保证是接口质量的一部分。并发三件套:`thread` 管理执行,`mutex/atomic` 保护数据,条件变量与 `async` 组织协作;数据竞争与死锁靠设计规避。下一章进入 C++ 最强大的部分:泛型与模板元编程。